Scope and Namespace in Functions

Python looks a name up in this order: Local → Enclosing → Global → Built-in (LEGB).

The LEGB rule

Scope	Where	Examples
Local	inside the current function	parameters, names assigned in the body
Enclosing	outer function(s) of a nested function	closure variables
Global	module (file) level	top-level names
Built-in	the `builtins` module	`len`, `print`, `range`

```
y = "global y"
def outer():
    z = "enclosing z"
    def inner():
        w = "local w"
        print(w, "|", z, "|", y, "|", len("abc"))
    inner()
outer()
```

OUTPUT

```
local w | enclosing z | global y | 3
```

Namespaces = name → object dictionaries

```
import builtins
count = 10
def f(a):
    b = a * 2
    # f's local namespace
    print(sorted(locals()))
f(1)
# module namespace
print("count" in globals())
# built-in namespace
print(hasattr(builtins, "len"))
```

OUTPUT

```
['a', 'b']
True
True
```

- A new local namespace is created on **every call** and discarded on return (so recursion is safe).
- `globals()`, `locals()` and `dir()` show which names exist.

The enclosing scope outlives the call

```
def make_multiplier(n):
    def times(x):
        # n comes from the enclosing scope
        return x * n
    return times

double = make_multiplier(2)
print(double(21))
print(double.__closure__[0].cell_contents)
```

OUTPUT

```
42
2
```

What Python does with a name

You write	Python does
`print(x)`	searches L → E → G → B; `NameError` if absent
`x = 1`	binds `x` in the **current** scope
`global x` / `nonlocal x`	sends assignments to that outer scope
`del x`	removes the name from the current scope

Assignment makes a name local

```
total = 0
def add(n):
    # assignment => `total` is local here...
    total = total + n
try:
    add(5)
except UnboundLocalError as e:
    # ...but read before it has a value
    print(e)
```

OUTPUT

```
cannot access local variable 'total' where it is not associated with a value
```

- **Reading** a global inside a function works.
- **Assigning** creates a new local, unless declared `global` / `nonlocal`.
- Mutating (`lst.append`) is not assignment – it works without a declaration.

Loops vs comprehensions

```
for i in range(3):
    pass
# loop variable survives
print(i)
squares = [k * k for k in range(3)]
# comprehension variable doesn't
print("k" in globals())
```

OUTPUT

```
2
False
```

PITFALL Shadowing a built-in (`list = [1]`, `sum = 0`, `id = 7`) silently breaks later calls to the real one.

TIP Keep functions pure: pass data in as parameters and return results, instead of leaning on globals.